Arab Academy for Science and Technology

College of Computing and Information Technology

Department of Computer Science

Grainola

AI-Powered Meeting Intelligence Platform

Web Engineering Course Project Report

Submitted By

Paula Akram Samir

Student ID: 221017681

1. Project Description

1.1 Overview

Grainola is a full-stack, AI-powered meeting intelligence platform inspired by tools like Grain.co and Otter.ai. The platform allows users to upload audio or video recordings of meetings, automatically transcribes them using Groq's Whisper model, generates structured AI summaries using Llama 3, and extracts key highlights. Users can organise meetings into folders, connect their Google Calendar for seamless scheduling, and search across all transcripts and summaries.

The application targets professionals, teams, and students who regularly attend meetings and need an efficient way to capture, review, and act on meeting content without manually reviewing hours of recordings.

1.2 Core Features

- Audio/video upload with direct-to-S3 presigned URL flow
- Automatic speech-to-text transcription via Groq Whisper API
- AI-generated meeting summaries (short + long-form) using Llama 3
- Highlight creation and management for key moments
- Folder-based meeting organisation with colour coding
- Global search across titles, transcripts, and summaries
- Google Calendar OAuth2 integration for importing scheduled meetings
- Tiered subscription system: Free, Basic ($9.99/mo), and Premium ($29.99/mo)
- Responsive, modern UI built with Next.js 14 and Tailwind CSS
- JWT authentication with secure refresh token rotation

1.3 Technology Stack

Layer	Technology	Purpose
Frontend	`Next.js 14 (App Router)`	React framework with SSR/CSR
Styling	`Tailwind CSS + shadcn/ui`	Utility-first responsive design
State	`TanStack React Query v5`	Server state & cache management
Auth State	`Zustand (persisted)`	Client auth store with localStorage
Backend	`Node.js + Express + TypeScript`	REST API server
Database	`MongoDB Atlas (M0 Free)`	Document database
ODM	`Mongoose`	MongoDB schema & query layer
Auth	`JWT (15 min) + Refresh Tokens`	Secure token-based authentication
Transcription	`Groq Whisper`	Fast speech-to-text
Summarisation	`Groq Llama 3`	AI text generation
File Storage	`AWS S3 + Presigned URLs`	Secure media uploads
Calendar	`Google Calendar API v3`	OAuth2 calendar integration
Validation	`Zod`	Runtime schema validation

2. System Architecture

2.1 High-Level Architecture

Grainola follows a decoupled, three-tier architecture separating the presentation layer (Next.js frontend), the application logic layer (Express API), and the data persistence layer (MongoDB Atlas). Third-party services are integrated at the application layer.

2.2 Component Diagram

FRONTEND (Next.js 14) React Query • Zustand • Tailwind Pages: Landing | Auth | Dashboard | Meetings | Pricing *Axios HTTP client with JWT interceptor*	**BACKEND (Express + TypeScript)** REST API • Zod Validation • JWT Auth Routes: Auth | Meetings | Folders | Subscription | Calendar *Fire-and-forget processing pipeline*
DATA LAYER (MongoDB Atlas) Collections: Users • Meetings • Recordings • Transcripts • Summaries • Highlights • Folders	**THIRD-PARTY SERVICES** Groq Whisper (transcription) Groq Llama 3 (summarisation) AWS S3 (media storage) Google Calendar API (integration)

2.3 Request Flow

- User authenticates via JWT; access token (15 min) stored in memory, refresh token (7 days) stored in MongoDB
- Frontend requests a presigned S3 URL from the backend
- Browser uploads the media file directly to AWS S3 (no server buffering)
- Frontend notifies backend of the completed upload
- Backend creates a Meeting document with status PENDING and returns immediately
- A background processor.service picks up the job: downloads from S3 → sends to Groq Whisper → stores transcript → sends to Groq Llama 3 → stores summary → updates status to COMPLETED
- Frontend polls (React Query refetch) until status changes to COMPLETED

2.4 Authentication & Security

- Short-lived JWT access tokens (15 minutes) minimise exposure window
- Refresh tokens are UUID v4, stored hashed in MongoDB with expiry
- Axios interceptor automatically refreshes expired access tokens transparently
- All sensitive routes are protected by auth middleware verifying the JWT signature
- Plan limits enforced server-side via requirePlan / attachPlanLimits middleware
- Zod schemas validate all incoming request bodies and query parameters

3. Database Schema

Grainola uses MongoDB Atlas (M0 free tier) as its primary datastore, accessed via Mongoose ODM. All collections use MongoDB ObjectIds as primary keys. The schema is designed for document-oriented storage with embedded subdocuments for closely related data and references for normalised relationships.

3.1 Users Collection

Field	Type	Required	Description
`_id`	ObjectId	Auto	Primary key
`email`	String	Yes	Unique email address
`password`	String	Yes	bcrypt hashed password
`name`	String	Yes	Full display name
`googleId`	String	No	Google OAuth ID for calendar
`googleTokens`	Object	No	OAuth access/refresh tokens
`subscription`	Object	Yes	Embedded subscription subdoc (plan, startedAt, expiresAt)
`refreshTokens`	Array	No	Active refresh token hashes
`createdAt`	Date	Auto	Mongoose timestamp

3.2 Meetings Collection

Field	Type	Required	Description
`_id`	ObjectId	Auto	Primary key
`userId`	ObjectId ref	Yes	Owner (ref: User)
`folderId`	ObjectId ref	No	Parent folder (ref: Folder)
`title`	String	Yes	Meeting title (max 200 chars)
`description`	String	No	Optional description
`meetingDate`	Date	Yes	When the meeting occurred
`durationSeconds`	Number	No	Duration from audio metadata
`status`	Enum	Yes	PENDING | PROCESSING | COMPLETED | FAILED
`calendarEventId`	String	No	Google Calendar event ID
`createdAt`	Date	Auto	Mongoose timestamp

3.3 Additional Collections

- Recordings — s3Key, s3Url, mimeType, fileSize, duration; belongs to one Meeting
- Transcripts — fullText, segments (array of {start, end, text, speaker}), language; belongs to one Meeting
- Summaries — shortText (300 chars), longText (2000 chars), keyPoints (array), actionItems (array), topics (array); belongs to one Meeting
- Highlights — text, startTime, endTime, note, tags; belongs to one Meeting
- Folders — userId, name, color (hex), description, position (display order); belongs to one User

3.4 Subscription Subdocument

- plan: Enum ("free" | "basic" | "premium"), default "free"
- startedAt: Date — when the current plan began
- expiresAt: Date — null for free tier, set on paid plans

Plan limits enforced at the API layer: Free (5 meetings/month, 30 min max), Basic (50 meetings/month, 2 hours max), Premium (unlimited).

4. API Endpoints

All endpoints are prefixed with /api/v1. Protected routes require a valid JWT access token in the Authorization: Bearer header.

4.1 Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/auth/register	Public	Register new user (name, email, password)
POST	/auth/login	Public	Login, returns access + refresh tokens
POST	/auth/refresh	Public	Exchange refresh token for new access token
POST	/auth/logout	JWT	Invalidate refresh token
GET	/auth/me	JWT	Get current authenticated user profile

4.2 Meetings Endpoints

Method	Endpoint	Auth	Description
GET	/meetings	JWT	List meetings (page, limit, folderId, search, status filters)
POST	/meetings	JWT	Create new meeting (title, folderId, meetingDate)
GET	/meetings/:id	JWT	Get single meeting with recordings, transcript, summary, highlights
PUT	/meetings/:id	JWT	Update meeting title, description, folder, date
DELETE	/meetings/:id	JWT	Delete meeting and all related data
PATCH	/meetings/:id/move	JWT	Move meeting to different folder
POST	/meetings/:id/highlights	JWT	Add a highlight to a meeting
DELETE	/meetings/:id/highlights/:hId	JWT	Remove a highlight

4.3 Recordings / Upload Endpoints

Method	Endpoint	Auth	Description
POST	/upload/presigned-url	JWT	Get S3 presigned PUT URL for direct browser upload
POST	/upload/complete	JWT	Notify backend upload is done; triggers async processing

4.4 Folders Endpoints

Method	Endpoint	Auth	Description
GET	/folders	JWT	List all folders for current user
POST	/folders	JWT	Create folder (name, color, description)
PUT	/folders/:id	JWT	Update folder name, color, or description
DELETE	/folders/:id	JWT	Delete folder (meetings moved to root)

4.5 Subscription Endpoints

Method	Endpoint	Auth	Description
GET	/subscription/plans	Public	Get all available plans (Free, Basic, Premium)
GET	/subscription/my-plan	JWT	Get current user subscription details
POST	/subscription/subscribe	JWT	Subscribe to a plan by planId
POST	/subscription/change-plan	JWT	Switch to a different plan

4.6 Google Calendar Endpoints

Method	Endpoint	Auth	Description
GET	/calendar/auth-url	JWT	Get Google OAuth2 authorisation URL
GET	/calendar/oauth/callback	Public	OAuth2 callback; stores tokens
GET	/calendar/events	JWT	Fetch upcoming calendar events
POST	/calendar/import/:eventId	JWT	Import calendar event as a meeting
DELETE	/calendar/disconnect	JWT	Revoke Google Calendar access

5. UI Screenshots

The following screenshots were taken from the live running application, demonstrating the core user interface flows.

5.1 Main Dashboard

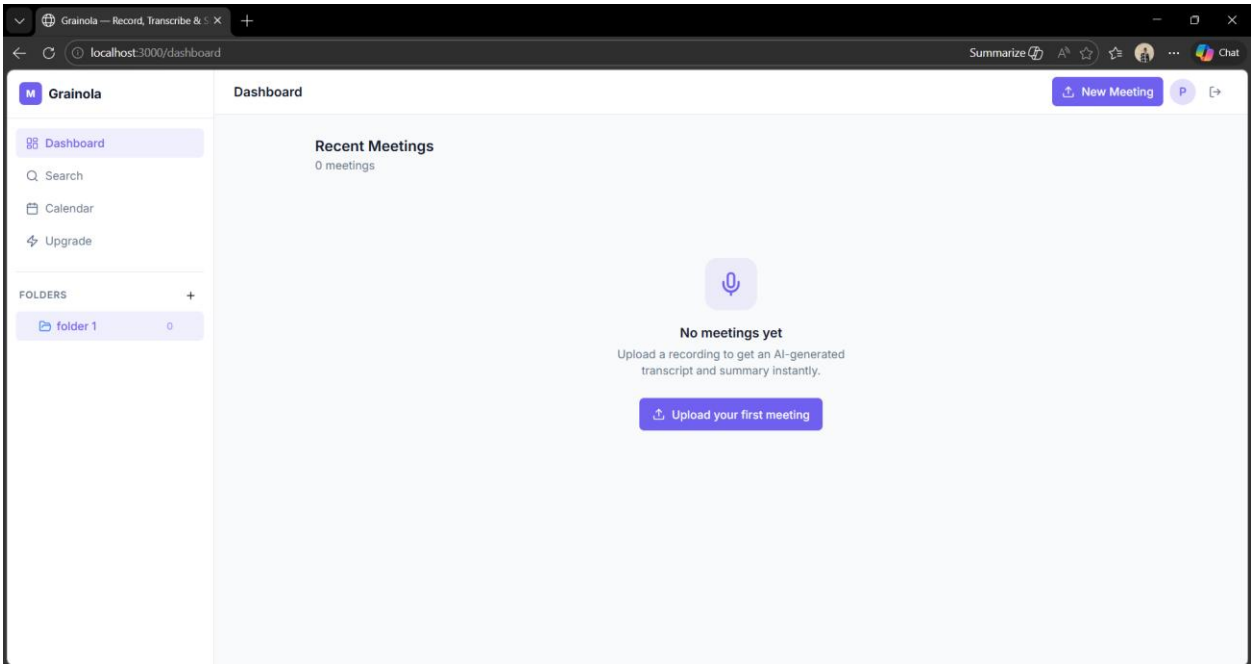

Figure 1: Main dashboard showing meeting list with status indicators and folder navigation

5.2 Authentication — Login

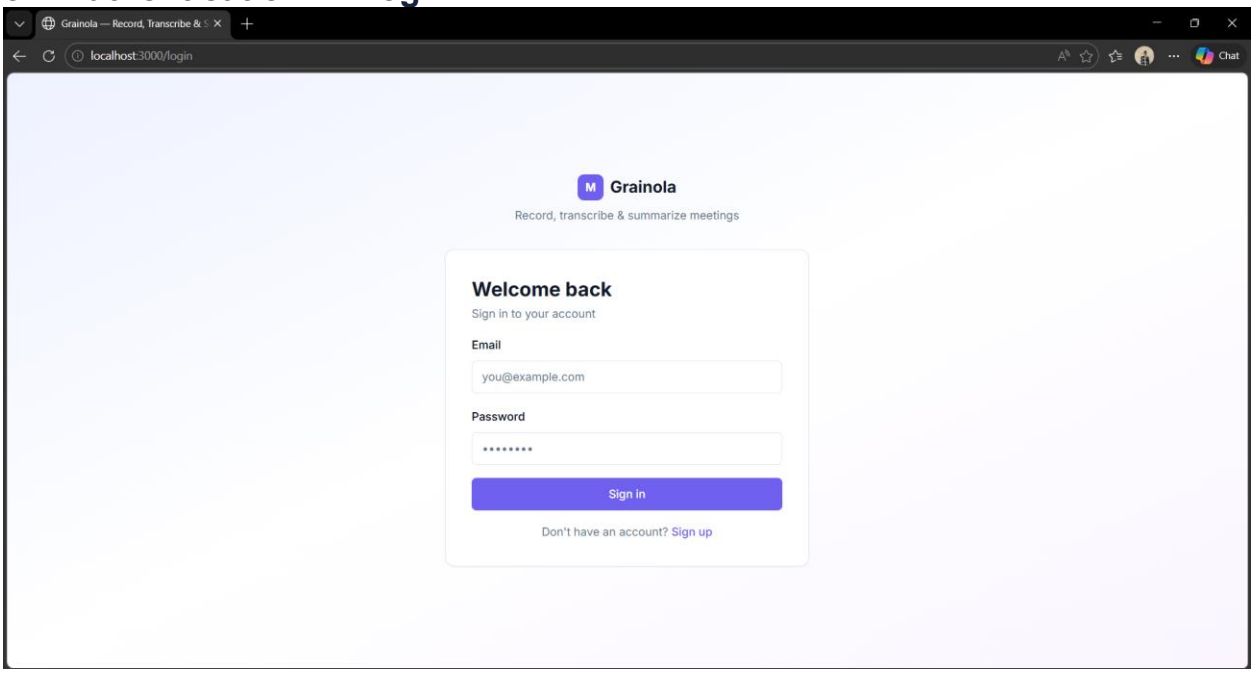

Figure 2: Login page with email/password form and link to registration

5.3 Authentication — Registration

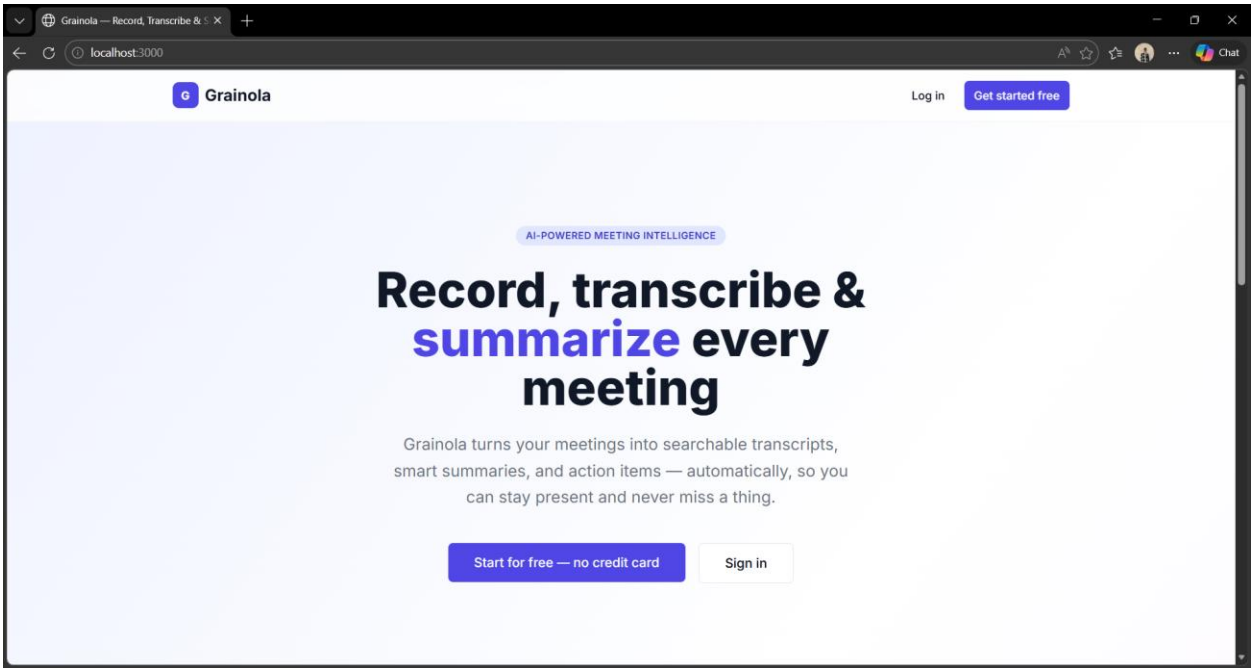

Figure 3: Registration page for new user sign-up

5.4 Meeting Detail View

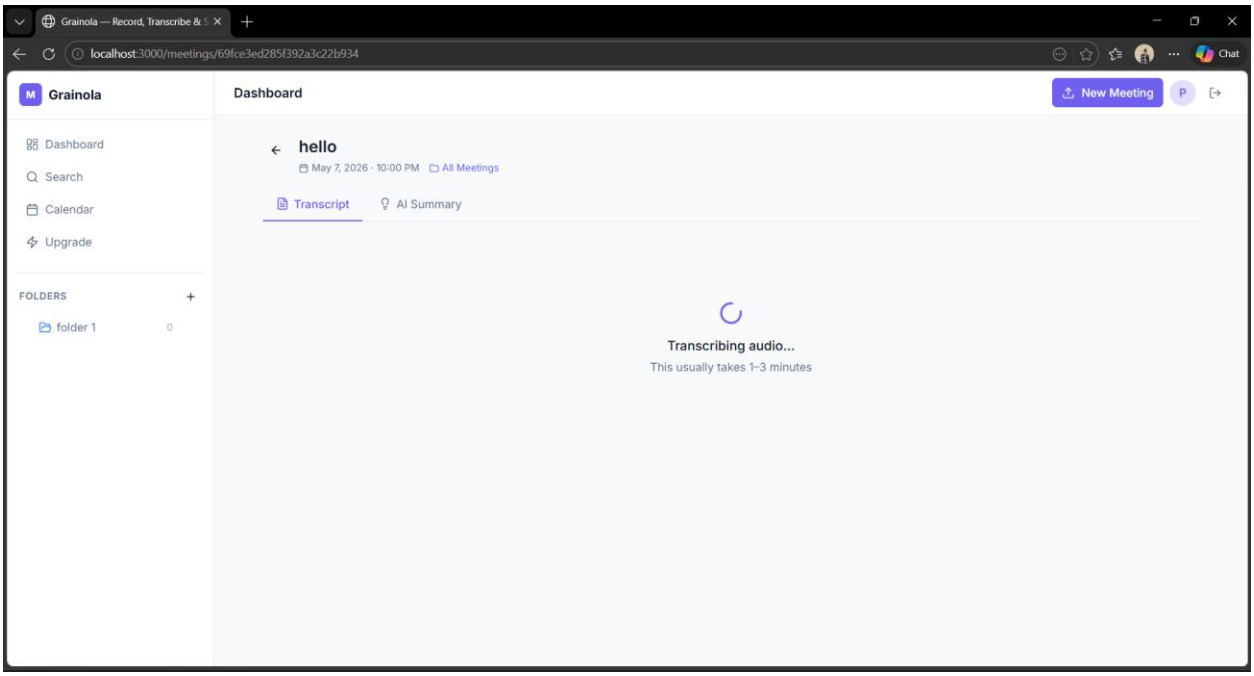

Figure 4: Meeting detail page showing Transcript and AI Summary tabs with processing state

5.5 Search Functionality

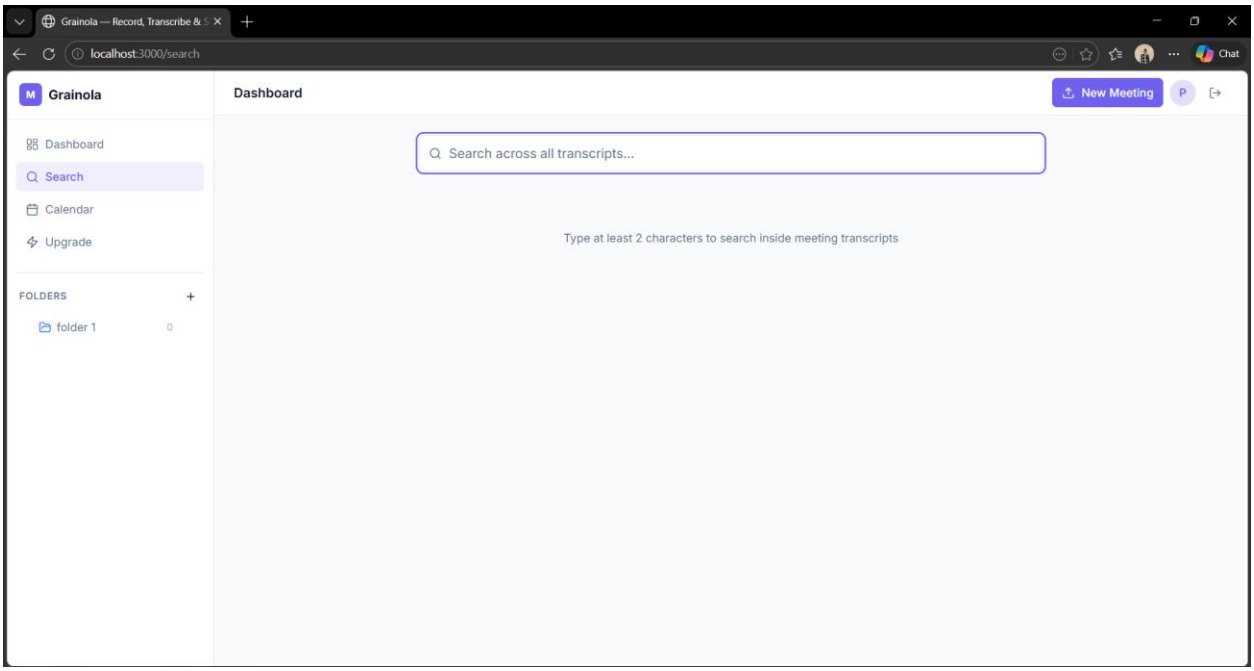

Figure 5: Global search across meeting transcripts and summaries

5.6 Google Calendar Integration

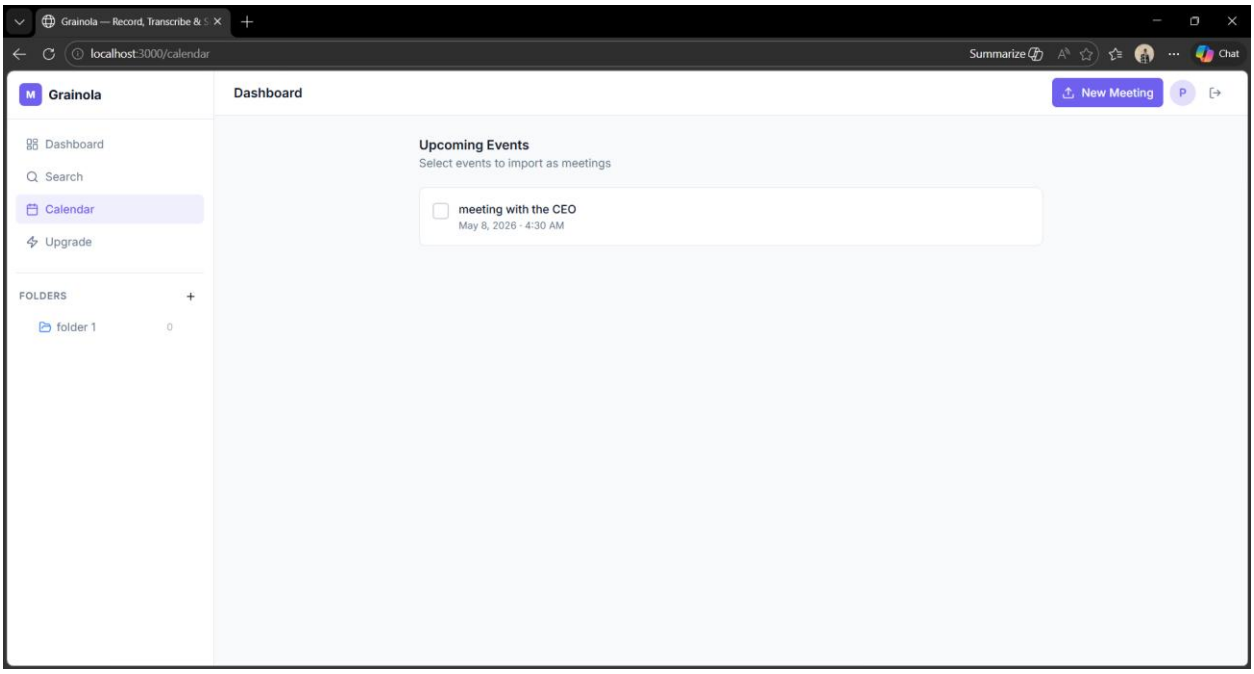

Figure 6: Google Calendar integration page for connecting and importing meetings

5.7 Pricing & Subscription Plans

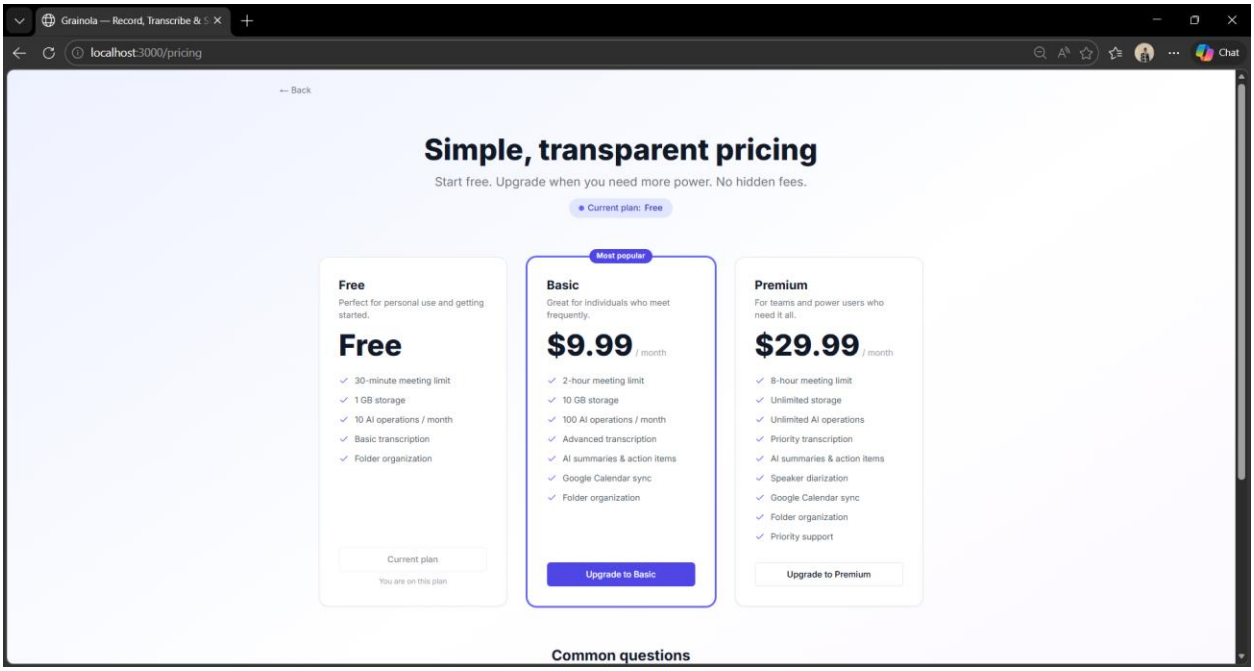

Figure 7: Pricing page with Free, Basic ($9.99/mo), and Premium ($29.99/mo) plan cards

6. Conclusion

Grainola successfully demonstrates a complete, production-grade web application built with modern technologies. The project integrates a Next.js 14 frontend with a Node.js/Express backend, MongoDB Atlas database, and multiple third-party APIs (Groq, AWS S3, Google Calendar) into a cohesive platform.

Key engineering challenges solved during development include: JWT token rotation with transparent refresh, MongoDB Atlas direct connection (bypassing SRV DNS resolution issues in Node.js), a fire-and-forget async processing pipeline for AI transcription and summarisation, MongoDB ObjectId validation (replacing incorrect UUID validators), and null-safe rendering for meetings without folder assignments.

The subscription system demonstrates plan-based access control implemented cleanly as Express middleware, keeping business logic separate from route handlers. The platform is designed to be extensible — adding new AI models, storage providers, or subscription tiers requires minimal changes.

All requirements specified in the Web Engineering course project brief have been fulfilled, including a responsive multi-page application, authenticated REST API, external service integration, and database persistence.